

CLAIMIT

Insurance Claims & Local Commerce Platform

Project Architecture & Scope Document

Version 1.0 | June 2026 | Prepared for Architect Review

3

Platform Components

20+

API Route Modules

16+

Mobile Features

3

Web Portals

Table of Contents

1. Executive Summary
2. Project Overview
3. Target Users
4. System Architecture
5. Model / Data Specification
6. Mobile Application Scope
7. Website (Web Portal) Scope
8. Scope Boundaries
9. Technology Stack
10. API Reference
11. Deployment & Infrastructure

12. Next Steps & Roadmap

1. Executive Summary

Claimit is a full-stack, multi-platform application merging insurance claims management with a hyperlocal commerce and advertising ecosystem. It delivers a Flutter-powered mobile app for consumers, a React web portal for businesses and sales agents, and a FastAPI + MongoDB backend deployed on Vercel and AWS.

The platform addresses two distinct but complementary markets: (1) individuals who need a streamlined, paperless way to file and track insurance claims, and (2) local businesses that want affordable digital advertising and footfall-driving deal campaigns targeted at nearby consumers.

At the intersection of these markets sits an AI-powered bill-reader: users scan retail purchase bills through the camera; Google ML Kit extracts merchant, amount, and line-items on-device; and the backend awards reward points redeemable at partner shops. This creates a loyalty loop that benefits consumers, merchants, and advertisers alike.

Mobile App	Flutter 3.x — cross-platform iOS & Android with real-time claim tracking, bill OCR, deal discovery, short-form video reels, and classifieds marketplace.
Web Portal	React 18 + Vite — three role-based portals: Advertiser, Sales/Affiliate, and Shop Owner with campaign management and analytics dashboards.
Backend API	FastAPI 0.115 — 20+ route modules, JWT authentication, async MongoDB via Motor, file upload, location services, reward engine, and admin panel.
AI / OCR	Google ML Kit Text Recognition (on-device) for bill scanning; server-side reward eligibility logic with configurable rules per merchant category.
Deployment	Backend on Vercel (primary) and AWS EC2 (fallback); MongoDB on Atlas; Flutter distributed via standard app stores; web via Firebase Hosting.

2. Project Overview

Claimit began as a pure insurance claims management application and evolved into a broader local commerce platform. The project is organised into three top-level directories inside a single monorepo:

Directory	Purpose	Language
claimitapporg/	Flutter mobile app — consumer-facing: claims, bill OCR, deals, reels, classifieds	Dart / Flutter
claimit_web_org/	Web portal — Advertiser, Sales & Shop Owner dashboards (React + Vite) + shared backend	JavaScript / Python
fastapi/	Shared backend API for mobile app — all domain routes, auth, file storage, admin	Python / FastAPI

Core Problem Statements

- Insurance policyholders have no single digital tool to file, document, and track claims end-to-end.
- Small local businesses cannot afford large-platform advertising; they need hyperlocal, affordable reach.
- Consumers lack incentive to explore local deals; a reward-for-spending loop bridges this gap.
- Disconnected portals for advertisers, sales agents, and shop owners create friction in the local-ads ecosystem.

Solution Architecture Summary

The platform is built on a hub-and-spoke model. The FastAPI backend is the hub, serving both the mobile app and web portals. Each spoke consumes the same REST API with role-based JWT tokens controlling access. Feature modules are domain-isolated in both Flutter (features/ directory) and the API (one Python router per domain), enabling independent iteration.

3. Target Users

Claimit serves four distinct user personas across mobile and web surfaces:

Consumer / Policyholder

Individuals aged 22-55 holding one or more insurance policies (health, motor, home). They want a single mobile app to file claims with evidence, track status, scan bills for rewards, and discover local deals.

- File & track insurance claims with document upload
- Scan purchase bills to earn reward points
- Discover hyperlocal deals & shops
- Watch short Reelz (local promotions)
- Buy/sell via Classifieds
- Receive push notifications for claim updates

Shop Owner / Merchant

Local business owners who want to register their shop, list offers, manage promotions, and view engagement. They use the Shop Owner web portal.

- Register shop with images & categories
- Create and manage deals / offers
- View dashboard statistics
- Manage ratings & reviews
- Receive affiliated walk-ins via the app

Advertiser

Brands and local businesses running digital ad campaigns. They choose from multiple ad formats and manage spend via the Advertiser portal.

- Choose ad type: Home Banner, Promo Reelz, Brand Deals, Nearby Deals
- Upload creatives and set budget
- View real-time campaign analytics
- Manage transaction history

Sales / Affiliate Agent

Field agents who onboard new shops into the network, earning commissions per registration.

- Register and onboard new shops
- View team and earnings hierarchy
- Access tutorial / training resources
- Track affiliate commissions

4. System Architecture

4.1 High-Level Overview

The system follows a layered client-server architecture with three client tiers sharing a single API layer backed by MongoDB Atlas.

```
[ Flutter Mobile App ] [ React Web Portal ] | | +----- REST/HTTPS -----+ |
[ FastAPI Backend ] Vercel / AWS EC2 | [ MongoDB Atlas ] claimit_db (6+ collections)
| +-----+-----+ [ File Storage ] [ External Services ]
uploads/ -> S3 (prod) Google ML Kit OCR SMS OTP Provider Google / Facebook OAuth
                Firebase Hosting (web CDN)
```

4.2 Component Responsibilities

Component	Responsibility	Location
Flutter App	UI rendering, local state, camera/OCR, file picking, push notifications, GoRouter navigation	claimitapporg/lib/
Provider Layer	App-wide state via ChangeNotifier (Auth, Claims, Dashboard, Deals, Shops, etc.)	lib/core/providers/
Dio HTTP Client	All API calls; JWT injection via interceptor; 401 triggers silent token refresh + retry	lib/core/network/
React Web App	Three role-based SPAs sharing Vite build; Context API for auth state; React Router v6	claimit_web_org/fron tend/src/
FastAPI Router Layer	20+ domain routers; each owns its models, DB queries, and business logic	fastapi/backend/app/ routes/
MongoDB Motor Layer	Async I/O to Atlas; Motor client pooled at startup	fastapi/backend/app/ database.py
JWT Middleware	Access + refresh token validation; token rotation on refresh	fastapi/backend/app/ utils/auth.py
OTP Service	Static OTP (123456) for dev; pluggable send_otp_sms() for production SMS provider	fastapi/backend/app/ utils/otp.py
File Storage	Local uploads/ served as static files; path stored in DB; designed for swap to S3/GCS in production	fastapi/backend/app/ main.py

4.3 Claim Submission Data Flow

Step	Action	Layer
1	User taps New Claim on mobile	Flutter UI
2	Multi-step form validates inputs; image_picker captures evidence	Flutter / Dart

Step	Action	Layer
3	POST /claims/create { type, description, policy_id, amount }	Dio HTTP
4	Auth middleware decodes JWT; extracts user_id from token	FastAPI JWT
5	Claim document inserted in MongoDB with status=pending	Motor / Atlas
6	POST /claims/{id}/documents uploads binary; stored in uploads/claims/	FastAPI static
7	Notification document inserted; push notification queued	MongoDB + Push
8	Dashboard stats recalculated on next GET /dashboard call	FastAPI aggregation

5. Model / Data Specification

MongoDB Atlas hosts the **claimit_db** database. All collections use MongoDB's native ObjectId as `_id`. Motor (async PyMongo) handles all DB I/O in the FastAPI layer.

Collection: users

Field	Type	Description
<code>_id</code>	ObjectId	Primary key
<code>phone</code>	str (unique)	10-digit mobile number — primary identifier
<code>email</code>	str (unique)	Email address
<code>name</code>	str	Full name
<code>provider</code>	str	Registration: phone / google / facebook
<code>avatar_url</code>	str	Path to uploaded avatar image
<code>is_verified</code>	bool	Phone OTP verification flag
<code>kyc_details</code>	dict	Aadhar, PAN, address (planned)
<code>created_at</code>	datetime	Account creation timestamp

Collection: claims

Field	Type	Description
<code>_id</code>	ObjectId	Primary key
<code>user_id</code>	ObjectId -> users	FK to claim owner
<code>claim_type</code>	str	health / motor / home / life / other
<code>status</code>	str	pending / under_review / approved / rejected / settled
<code>description</code>	str	Incident description
<code>policy_id</code>	ObjectId -> policies	Linked policy (optional)
<code>amount</code>	float	Claimed amount
<code>documents</code>	list[str]	File paths of uploaded evidence
<code>timeline</code>	list[dict]	Status change history with timestamps
<code>created_at</code>	datetime	Submission timestamp

Collection: notifications

Field	Type	Description
_id	ObjectId	Primary key
user_id	ObjectId -> users	Recipient
title	str	Notification headline
message	str	Body text
type	str	claim_update / reward / deal / system
is_read	bool	Read state flag
created_at	datetime	Creation timestamp

Collection: policies

Field	Type	Description
_id	ObjectId	Primary key
user_id	ObjectId -> users	Policy holder
policy_number	str	Insurer policy number
insurer_name	str	Insurance company name
policy_type	str	health / motor / home / life
premium_amount	float	Annual premium
coverage_amount	float	Sum assured
start_date / end_date	datetime	Policy validity period

Collection: otp_store

Field	Type	Description
_id	ObjectId	Primary key
phone	str	Mobile number OTP was sent to
otp	str	6-digit code (hashed in production)
attempts	int	Verification attempts (max 3)
expires_at	datetime	TTL-indexed; auto-deleted after 10 min

Collection: shops

Field	Type	Description
_id	ObjectId	Primary key
owner_id	ObjectId -> users	Shop owner
name / category	str	Shop name and business type
location	GeoJSON Point	Lat/Lng for proximity queries
images	list[str]	Photo paths
offers	list[dict]	Active deals / promotions
rating	float	Aggregated customer rating
is_active	bool	Visibility flag

6. Mobile Application Scope

The Flutter mobile application (**claimitapporg/**) targets Android and iOS, with web support via **flutter run -d chrome**. The app is structured around feature modules, each self-contained with its own screens, models, providers, and widgets.

Feature Module	Scope Description	Pri orit y
Authentication	Splash, 3-screen onboarding, register, login, OTP (123456 static for demo), social login (Google & Facebook), profile completion, location permission gate.	Hig h
Dashboard	Stats overview: total claims, active policies, reward points, pending claims. Quick action tiles. Recent claims list. Shimmer loading skeleton.	Hig h
Claims Management	Claims list with status badges. Multi-step new-claim form. Document upload (camera or gallery). Claim detail with interactive timeline and status history.	Hig h
Bill Reader / OCR	Intro screen. Live camera viewfinder with flash toggle and scan animation. Google ML Kit on-device OCR extracts merchant name, date, total. Confirm screen. Reward points awarded. Success / wallet / review-pending screens.	Hig h
Deals & Offers	Browse deals list. Deal detail card (merchant, discount, expiry, terms). Filter by category. Location-aware sorting.	Me diu m
Shop Discovery	Shop list with category filter. Shop detail with images, offers, rating. Redeem flow: eligibility check to loading to success screens.	Me diu m
Reelz (Short Videos)	Vertical full-screen video feed for local brand promotions. Video player controls. Swipe navigation.	Me diu m
Classifieds Marketplace	Home with category grid. Listing detail with image carousel. Add-post multi-step flow (category, photos, details, review). My posts management.	Me diu m
Notifications	Full notification list. Read / unread state. Mark all as read. Notification badge on home tab.	Me diu m
Insurance Policies	Policy list. Add policy form (insurer, type, number, premium, coverage, dates). Policy detail screen.	Me diu m
User Profile	View profile with avatar. Edit profile (name, email, DOB, address). Avatar upload. Settings screen (logout, app version).	Me diu m
Location Onboarding	Permission request. GPS acquisition. Stored for proximity-based deal / shop sorting.	Lo w
Global Search	Cross-domain search across shops, deals, and classifieds.	Lo w

Feature Module	Scope Description	Priority
National Ads	Full-screen ad viewer for nationally targeted campaigns from the banner API.	Low
Category Browser	Grid of insurance and commerce categories for navigating to relevant lists.	Low

6.1 Key Technical Choices — Mobile

GoRouter 17.x	Declarative URL-based routing with deep-link support; shell route wraps the 5-tab bottom nav bar.
Provider (ChangeNotifier)	Lightweight state management; one provider per feature domain; injected at MaterialApp root.
Dio + Interceptor	All HTTP via Dio; request interceptor injects Bearer tokens; 401 triggers silent refresh then retry.
google_mlkit_text_recognition	On-device OCR; no API key required; works offline; supports multi-script text.
flutter_secure_storage	JWT tokens in OS keychain (iOS Keychain / Android EncryptedSharedPreferences).
geolocator + geocoding	GPS for deal/shop proximity sorting; reverse geocoding for human-readable address display.
isInitializing guard	Auth flag prevents flash-of-wrong-route before token validation completes on app launch.

7. Website (Web Portal) Scope

The web application (**claimit_web_org/frontend/**) is a React 18 + Vite SPA deployed via Firebase Hosting. It exposes three distinct portals under a single domain, each with its own auth flow, dashboard, and feature set.

L1 — Advertiser Portal

Route: /advertiser/auth

Brands and businesses running paid ad campaigns on the Claimit platform.

- Register with phone + email + OTP
- Choose Service: direct advertiser or via agency
- Choose Ad Type: Home Banner, Promo Reelz, Brand Deals, Nearby Deals
- Ad Details: creative upload, copy, target area, budget, duration
- Payment Page: pricing tiers and checkout
- Publish Ad: final review and go-live
- Advertiser Dashboard: impressions, clicks, spend, CTR
- Transaction History: all billing records

L2 — Sales / Affiliate Portal

Route: /sales/auth

Field agents who recruit and onboard new shop owners into the network.

- Register / login with phone OTP
- Sales Dashboard: shops added, earnings, commissions
- Add Shop: initiate shop onboarding on behalf of merchant
- Team View: hierarchy of recruited agents
- Tutorial Zone: training materials
- Incentive program and commission details

L3 — Shop Owner Portal

Route: /shop/auth

Merchants managing their shop profile, offers, and engagement analytics.

- Register with phone OTP
- Shop Registration: name, category, location, photos
- Choose Shop Type and categories
- Upload multiple shop images
- Review and Submit for approval
- Shop Dashboard: views, redemptions, ratings
- Manage Offers: create and edit active deals
- View Ratings & Reviews from app users

8. Scope Boundaries

Clear delineation of what is built, what is planned, and what is out of scope:

Feature / Capability	Status	Notes
Insurance claims CRUD + document upload	IN SCOPE	Fully built; demo mode active
OTP authentication (phone + email)	IN SCOPE	Static OTP 123456 in dev
Social login (Google, Facebook)	IN SCOPE	OAuth; keys via dart-define
Bill scanner (camera + on-device OCR)	IN SCOPE	Google ML Kit Text Recognition
Reward points earn / redeem flow	IN SCOPE	Basic flow implemented
Deal & shop discovery	IN SCOPE	List + detail + filter screens
Short-video Reelz feed	IN SCOPE	Vertical scroll video player
Classifieds marketplace	IN SCOPE	Post, browse, add listings
Advertiser web portal	IN SCOPE	Campaign creation + dashboard
Sales / Affiliate web portal	IN SCOPE	Shop onboarding + earnings
Shop Owner web portal	IN SCOPE	Registration + dashboard
Admin panel backend routes	IN SCOPE	Routes exist; admin UI pending
Push notifications (FCM)	PLANNED	Notification model exists; FCM integration pending
Real SMS OTP (Twilio / MSG91)	PLANNED	send_otp_sms() hook exists; provider not wired
KYC validation (Aadhar / PAN)	PLANNED	Field exists; validators needed
Payment gateway (Razorpay / Stripe)	PLANNED	Payment UI exists; gateway not integrated
HTTPS enforcement for all environments	PLANNED	Must be done before production launch
AI-assisted claim auto-assessment	FUTURE	Not yet designed — Phase 2
In-app video calls with agent	FUTURE	Phase 2
Multi-language (i18n)	FUTURE	Phase 2 — Hindi, Tamil, Telugu priority
Blockchain claim verification	OUT OF SCOPE	Not planned for this product
Direct insurer API integration	OUT OF SCOPE	Manual workflow only for now

9. Technology Stack

Mobile — Flutter / Dart

Technology	Version	Purpose
Flutter	3.19+	Cross-platform UI framework (iOS, Android, Web)
Dart SDK	^3.11.5	Language runtime
GoRouter	17.x	Declarative URL routing with deep-link support
Provider	6.x	App-wide state management (ChangeNotifier)
Dio	5.x	HTTP client with JWT interceptor and error handling
flutter_secure_storage	9–10.x	Keychain / EncryptedSharedPreferences for JWT tokens
google_mlkit_text_recognition	0.13.x	On-device OCR for bill scanning
image_picker	1.0.7 (pinned)	Photo capture and gallery access
file_picker	8.0.0 (pinned)	Document selection for claim uploads
camera	0.11.x	Live camera viewfinder for bill scanner
geolocator / geocoding	13.x / 3.x	GPS and reverse geocoding
lottie	3.x	JSON animation playback (splash, success screens)
video_player	2.x	Reelz vertical video feed
speech_to_text	7.x	Voice input for search and claim description
google_sign_in / facebook_auth	Latest	Social OAuth (Google & Facebook)
smooth_page_indicator	2.x	Onboarding dot indicators

Web Frontend — React

Technology	Version	Purpose
React	18	UI library
Vite	Latest	Build tool and HMR dev server
React Router	v6	Client-side SPA routing
Context API	Built-in	Auth and app-wide state across portals
Firebase Hosting	Latest	Static web hosting + CDN

Backend — Python / FastAPI

Technology	Version	Purpose
FastAPI	0.115+	ASGI web framework with automatic OpenAPI docs
Motor	3.x	Async MongoDB driver for Atlas
python-jose	3.x	JWT encode / decode
passlib [bcrypt]	1.7.x	Password hashing
python-multipart	Latest	File upload parsing
pydantic	v2	Request / response schema validation
uvicorn	Latest	ASGI server for local dev and EC2

Database & Storage

Technology	Version	Purpose
MongoDB Atlas	Cloud (M0 free tier)	Primary datastore — all collections
Motor (async)	3.x	Async I/O to Atlas; connection pooled at startup
Local uploads/	Filesystem	Avatars, claims, shop images (swap to S3 in prod)

Infrastructure & DevOps

Technology	Version	Purpose
Vercel	Serverless	Primary FastAPI deployment via vercel.json ASGI config
AWS EC2	Fallback	IP: 13.63.176.113 — manual uvicorn; port 8001
Firebase Hosting	CDN	React web portal static assets
GitHub	Version control	Monorepo — branches: main, sk
Gradle	8.10.2 (pinned)	Android build system (8.14 had cache corruption bugs)
AGP / Kotlin	8.7.0 / 1.9.24	Pinned for compatibility with image_picker, file_picker
Xcode	15+	iOS builds; CocoaPods managed dependencies

External APIs / SDKs

Technology	Version	Purpose
Google ML Kit	On-device	Text recognition for bill OCR — no API key, works offline
Google OAuth 2.0	Cloud	Social login — client ID passed via --dart-define
Facebook Login SDK	Cloud	Social login — app ID via --dart-define
SMS Provider (TBD)	Planned	Twilio / MSG91 for production OTP delivery
Razorpay / Stripe	Planned	Payment gateway for advertiser campaign billing
Firebase Cloud Messaging	Planned	Push notification delivery for mobile

10. API Reference

Base URL (production): <https://claimitorgbackend.vercel.app> | Swagger UI: </docs> | ReDoc: </redoc>

Method	Endpoint	Description	Auth
POST	/auth/send-otp	Send OTP to phone number	No
POST	/auth/verify-otp	Verify OTP; returns JWT access + refresh tokens	No
POST	/auth/register	Register new user (phone, email, name)	No
POST	/auth/login	Login existing user	No
POST	/auth/refresh	Rotate refresh token to new access token	No
POST	/auth/social/login	Social OAuth login (Google / Facebook)	No
POST	/auth/logout	Invalidate refresh token	Yes
GET	/users/profile	Get authenticated user profile	Yes
PUT	/users/profile/update	Update profile fields	Yes
POST	/users/avatar	Upload avatar image	Yes
POST	/users/profile/phone	Add phone to social-login account	Yes
GET	/claims	List all claims for authenticated user	Yes
POST	/claims/create	Create new insurance claim	Yes
GET	/claims/{id}	Get claim detail including timeline	Yes
POST	/claims/{id}/documents	Upload evidence document to claim	Yes
PATCH	/claims/{id}/status	Update claim status (admin only)	Yes
GET	/dashboard	Get aggregated dashboard statistics	Yes
GET	/notifications	List user notifications	Yes
PATCH	/notifications/{id}/read	Mark notification as read	Yes
GET	/policies	List user insurance policies	Yes
POST	/policies	Add new insurance policy	Yes
GET	/deals	List deals with optional location filter	Yes
GET	/deals/{id}	Get deal detail	Yes
GET	/shops	List shops by category / proximity	Yes
GET	/shops/{id}	Get shop detail with offers	Yes
POST	/shops	Register new shop (sales agent)	Yes
POST	/bill/scan	Process OCR result; award reward points	Yes
GET	/rewards	Get reward point balance and history	Yes
POST	/redeem	Redeem points at partner shop	Yes

Method	Endpoint	Description	Auth
GET	/reels	List Reelz short videos	Yes
GET	/classifieds	List classified posts	Yes
POST	/classifieds	Create classified listing	Yes
GET	/banners	Get active ad banners	Yes
GET	/locations	Get location reference data	Yes
GET	/admin/stats	Platform-wide admin statistics	Yes (admin)

11. Deployment & Infrastructure

11.1 Current Deployment

Component	Platform	URL / Details
FastAPI Backend	Vercel (serverless ASGI)	https://claimitorgbackend.vercel.app
FastAPI Backend (fallback)	AWS EC2	http://13.63.176.113:8001 — manual uvicorn
MongoDB	MongoDB Atlas (M0 free)	claimit_db — Atlas cluster
React Web Portal	Firebase Hosting	firebase.json in claimit_web_org/frontend/
Flutter Android	APK / Play Store	AGP 8.7.0, Kotlin 1.9.24, Gradle 8.10.2
Flutter iOS	Xcode / App Store	CocoaPods managed; Xcode 15+
Flutter Web	Browser	<code>flutter run -d chrome</code> (dev / demo)

11.2 Known Build Issues Fixed

Issue	Resolution
Gradle 8.14 cache corruption	Pinned wrapper to 8.10.2; fix_build.bat clears corrupt cache
AGP 9 / Kotlin 2.2 incompatibility	Pinned AGP 8.7.0, Kotlin 1.9.24; image_picker 1.0.7, file_picker 8.0.0
Android cleartext HTTP blocked	network_security_config.xml whitelisting 10.0.2.2 + localhost
iOS ATS blocking local API	Info.plist NSAllowsArbitraryLoads + NSAllowsLocalNetworking
Flutter web 401 redirect loop	Dio validateStatus = status < 500; providers read error body
Login flicker on cold start	isInitializing flag in AuthProvider gates splash screen

11.3 Production Hardening Checklist

- ☐ Replace CORS allow_origins=[*] with specific frontend domain(s)
- ☐ Serve API over HTTPS only; disable cleartext HTTP on Android
- ☐ Wire real SMS provider (Twilio / MSG91) into backend/app/utls/otp.py
- ☐ Remove dev_otp from OTP register API response
- ☐ Integrate Razorpay or Stripe for advertiser payments
- ☐ Add Aadhar/PAN format validators for KYC module
- ☐ Integrate Firebase Cloud Messaging for push notifications
- ☐ Migrate uploads/ to AWS S3 or Google Cloud Storage
- ☐ Add RBAC middleware to restrict admin routes
- ☐ Add rate limiting on auth endpoints to prevent brute-force

12. Next Steps & Roadmap

Phase 1 — Production Readiness (Immediate)

- Integrate real SMS OTP provider (Twilio / MSG91)
- Enforce HTTPS across all environments
- Restrict CORS to production domains
- Firebase Cloud Messaging (FCM) push notifications
- Migrate file storage to AWS S3 or GCS
- Admin dashboard UI for claim status management
- Android Play Store submission; then iOS App Store

Phase 2 — Feature Expansion (1–3 months)

- KYC module: Aadhar/PAN validation + document upload
- Razorpay payment gateway for advertiser billing
- Advanced reward engine: tiered points, expiry, bonus categories
- Advertiser analytics: impressions, clicks, conversions, CTR
- Geo-fencing: trigger deals/notifications near partner shops
- Multi-language support (Hindi, Tamil, Telugu as priority)
- Insurer partner API: direct claim status sync

Phase 3 — AI & Scale (3–6 months)

- AI claim assessment: document analysis + fraud risk scoring
- Personalised deal recommendations based on bill scan history
- In-stream Reelz video ads with CPM / CPC billing
- Real-time chat: consumer to insurance agent / shop support
- B2B API: white-label claims module for insurer partners
- Microservices migration for high-traffic domains (deals, reels)
- CI/CD pipeline with GitHub Actions and automated test suite

This document represents the architecture and scope of Claimit as of June 2026. It is intended for architect review and should be treated as a living document. Update it as features are completed or design decisions change. Contact: sgramesh1980@gmail.com